

Project Intelligence Hub

How the App Works & Its RAG Pipeline — a plain-language guide
Retrieval-Augmented Generation over Blend360 project documents

1. What the app does

The Project Intelligence Hub (PIH) turns ~900 scattered Blend360 project documents (PowerPoint, Word, PDF, Excel) into a searchable, AI-powered knowledge base. Sales, delivery, and leadership teams can ask natural-language questions and get answers drawn **only** from the real documents (with source citations), filter the corpus by technology / industry / year, and auto-generate one-page sales briefs.

It uses **RAG (Retrieval-Augmented Generation)**: instead of asking an LLM to answer from memory, the app first *retrieves* the most relevant document chunks, then asks the LLM to answer *using only those chunks*. This keeps answers grounded and avoids hallucination.

2. The AI models used

Role	Model	What it does
Embeddings	text-embedding-3-large (OpenAI)	Converts text into a 3072-dimensional vector. Used for both document chunks and
LLM (generation)	gpt-5.6-terra (primary), gpt-5.2 (fallback)	Writes the grounded answer / sales brief. Temperature 0 = deterministic.

Model resolution: at startup the app checks which models the API key can actually call and falls back automatically, so a renamed or unavailable model never crashes it.

3. Indexing — building the vectors (done once, offline)

Before any search, every document is processed into vectors and saved to a single pickle file. This is the **embed_docling** step:

- **Parse** each file with **Docling** into a structured document.
- **Chunk** it with Docling's HybridChunker (token-aware, max ~8191 tokens), preserving each chunk's page/slide numbers and heading trail.
- **Embed** each chunk's text with text-embedding-3-large → a 3072-float vector.
- **Store** everything as one pickled Python list — each element is a dict holding the chunk text, its embedding vector, and rich metadata.

Each record in the pickle looks like this:

```
{
  'doc_id': 'NCL_Proposal.pptx',
  'text': 'the chunk text ...',
  'embedding': [0.013, -0.047, ... 3072 floats ...],    # the vector
  'source_ref': 'NCL_Proposal.pptx - slides 2-3',
  'page_nos': [2, 3], 'headings': ['Feature Store'],
  'technologies': ['Snowflake'], 'industries': ['Travel'], 'year': 2026, ...
}
```

Current index: **845 documents** → **2,161 chunk records**, in one ~60 MB file
(data/embeddings/docling_records.pkl).

4. Search from vectors using the pickle (the core)

There is **no database and no special index** — search is exact nearest-neighbor done with a single matrix multiply in NumPy. Four steps:

Step A — Load the pickle once. build a normalized matrix

```
_records = pickle.load(f)                # the list of dicts
M = np.asarray([r['embedding'] for r in _records])  # shape (2161, 3072)
M = M / np.linalg.norm(M, axis=1, keepdims=True)  # unit-length each row
```

All 2,161 vectors are stacked into one matrix M and each row normalized to length 1. This is cached in memory, so the file is read from disk only once.

Step B — Embed the query the same way

```
v = embeddings.create(model='text-embedding-3-large', input=[q]).data[0].embedding
v = v / np.linalg.norm(v)                # also unit length
```

Step C — Cosine similarity = one matrix multiply

```
scores = M @ v                # (2161, 3072) . (3072,) -> (2161,) similarity scores
```

This is the key idea: the cosine similarity of two unit vectors is just their dot product. Because both the matrix rows and the query are normalized, **M @ v** computes the similarity of the query against *all 2,161 chunks at once* in one fast operation. `scores[i]` = how relevant chunk *i* is to the question.

Step D — Hybrid boost, filters, and top-k

- **Hybrid boost:** a small keyword (IDF) score is added so rare query words (e.g. '61', 'Vialto') pull in chunks that literally contain them, even if their embedding is weak. Kept small (0.25) so it nudges, never overrides, the semantic ranking.
- **Filters:** non-matching chunks (wrong technology/industry/year) are set to minus-infinity so they can't be picked — this powers the UI filter panel.
- **Top-k:** `np.argsort` grabs the *k* highest-scoring chunks (default 6) and returns them with their metadata and score.

That is the entire 'vector database': a pickle load + one **M @ v** + a partial sort.

5. Generation — a grounded, cited answer

- The top-k chunks are formatted into a context block, each tagged with its `source_ref` (file + slide/page).
- That context + the question + a strict system prompt ('answer ONLY from this context, cite sources, say so if you can't find it') go to gpt-5.6-terra at temperature 0.
- The model returns a cited answer; the app also returns a de-duplicated list of sources with page/author/technology metadata.

This is what makes answers **grounded** (from your documents, with citations) and **honest** (it declines rather than inventing when the fact isn't indexed).

6. Why a pickle file instead of a vector database?

- **Simplicity** — one file, no server, no schema; install and go.
- **Speed** — brute-force cosine over ~2,000 vectors is faster than the overhead of a specialized index at this scale (one matrix multiply is microseconds).
- **Rich metadata** — each record is a plain Python dict, so filtering, faceting, and sales-brief source-gathering are trivial.

- **Portability** — the whole knowledge base is one file you can copy anywhere.

Trade-off: brute-force is linear in the number of chunks, so at ~100k+ chunks you'd switch to an approximate index (FAISS / Chroma). The app also ships a Chroma-backed backend for exactly that path — same embed-store-cosine idea, but the database manages the index.

In one sentence

Every document chunk is embedded into a 3072-dimension vector with text-embedding-3-large and pickled as a list of dicts; at query time those vectors load into one normalized NumPy matrix so cosine search is a single matrix multiply, and the best-matching chunks are handed to gpt-5.6-terra with a strict 'answer only from these sources' prompt.

Generated for the PIH Hackathon — The Curators